The `which` command searches the various file system paths stored in the shell's environment and reports back with the absolute path if the `php` binary is found. Another option would be to simply invoke PHP, like this:

```
$ php -v
```

```
Output:
PHP 8.4.5 (cli) (built: Mar 12 2025 01:55:56) (NTS)
Copyright (c) The PHP Group
Built by Homebrew
Zend Engine v4.4.5, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.5, Copyright (c), by Zend Technologies
```

The precise output will vary but should give you the PHP version (8.4.5 in the example output above) and the *Server API* (SAPI) in parentheses. SAPI is shorthand for the various mechanisms that enable interaction between PHP and the rest of the system which is running it, such as file or network data streams, environment variables, HTTP data structures, and so on. The SAPI here is "cli" which means it is the standalone system binary that is being invoked. The CGI binary equivalent to `php` is usually named `php-cgi` and is also invokable from a shell.

```
$ php-cgi -v
```

```
Output:
PHP 8.4.5 (cgi-fcgi) (built: Mar 12 2025 01:55:56) (NTS)
Copyright (c) The PHP Group
Built by Homebrew
Zend Engine v4.4.5, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.5, Copyright (c), by Zend Technologies
```

Notice that the SAPI part of the output is now "cgi-fcgi", indicating the server API is now PHP's "FastCGI" interface. If `php-cgi` isn't available on your system, try `$ php-fpm -v`. If neither is installed, then don't worry; I cover installation next. There's a little bit more work to do to get PHP running via a web-server, and we'll get to that too. First let's deal with what happens if PHP isn't installed on your system.